
Your ebook has decrypted.

The TXT file just save 20% of ebook content.

Please register the software, then get all contents and the valid ebook format.

Title : Coding NestFlow: Fluent Box-Scoped JavaScript and Wasm

Author: Harper, Gary

Coding NestFlow

Fluent box-scoped

JavaScript && Wasm

gary-harper.com

Institute for Nagging Doubt

Puerto Princesa, Filipinas

© 2021 Gary Harper

All rights reserved

Index of ContentsPreface

Flowing Expression

Design musing—always returning something

Design musing—concatenation

Design musing—bitwise not

Design musing—decorating verbs

Final design musing—argStack

Box-nested namespaces

Edit-debug: basics

Design musing—fixName vs varName

Design musing—whitespace

Design musing—inName vs outName

A picnic

Design musing—declaration-by-assignment

Name interconnections give thought

With a heap, names are cheap

Skipping Around

Design musing—conditional exit

Design musing—if transparency

Becoming Functional

Becoming Objective

Well-Dimensioned Numbers

Fixing Nothing

Zero: birth and adolescence

Zero: mature numberhood

Tunnel-vision evolution

N: Context-adaptive paradise

N: Context-specific purgatory

N: Container hell

Final Thoughts

Skipping Around

Becoming Functional

Becoming Objective

Playing Around

PrefaceThe engine in browsers is fast becoming the universal virtual machine for apps; and its coding is steadily evolving toward transparency and efficiency. Advancing further toward those two goals is the aim of NestFlow. It is currently implemented in JavaScript—but clunkily owing to historical infelicities—so it is intended to eventually be implemented mostly in Wasm. That idiom is widely hoped and expected to become the universal low-level bytecode for browsers.

And now a short message from your author:

This book is not finished; but I am flinging it out now because my expiration date is approaching. Unexpectedly, hundreds of readers downloaded it in the first few days; so I am embarrassed that I neglected to tidy it up first.

But it is now tidy: the prose has been revamped sentence by sentence; fuller explanations have been added; and the Final Thoughts chapter is greatly expanded. Most important is a large new Fixing Nothing chapter, which I

consider the most crucial part of this book. The ideas therein have been percolating in my mind about thirty years.

Readers who bought the shorter untidy version can get the expanded tidy version by clicking on “update available” in their Kindle Content on Amazon—that is the great advantage of a Kindle eBook over pirate download: the ideas within it will get polished.

The ideas herein enchant me; and my hope is that they will also enchant a team of competent programmers, who might like to finish turning them into a truly transparent and efficient language. NestFlow’s .js .css .html files can be found on github—search for “NestFlow” and click on YiAnIam/ Seasoned programmers are welcome to play with this code. Other programmers may need some seasoning to avoid bafflement.

I own the nestflow.net domain for way beyond my expiry, I believe; and would be happy to donate it to a team who manifests more expertise with JavaScript, Wasm and the DOM than I do. Contact gary_harper@mac.com And now back to our main, ahem, programming:

NestFlow embodies two simple ideas: flowing expression thru box-nested namespaces. The flow is made transparent to the programmer, but efficient for the machine by a simple branching discipline: a forward branch is nothing more than an explicit conditional skip over the next namespace box; a backward branch is merely an explicit tail call to its same-namespace box. That’s it.

This branching discipline allows you to express all the traditional control structures—if, else, else-if, while, do-while, for, switch ... and even the so-called “short-circuiting” logical operators and the ternary expression—in a computationally transparent way.

Using them, you will generate nearly pre-parsed code, something that has been self-contradictory until recently. The recent EcmaScript specs finally make this possible via enhanced literals, let and const scoping, pure lexically-scoped arrow functions; and best of all, mandatory tail-call optimization—JavaScript has finally achieved the capabilities of Scheme; but much enhanced with arrays, and objects having simple prototype inheritance.

NestFlow aims to streamline those enhancements, literally, like so:

Computation, by its very nature, flows at the hardware level; but expression of that flow has historically been tedious and opaque. Consider, for example, computing $(3+4)*5$. To get the flow started, addition's arguments must first be specified. This was done in the early machines by loading an Accumulator register A with 3, and a Data register D with 4, like so: LOAD A, 3; LOAD D, 4.

Next D was added to A: ADD A, D; which left 7 in A. To multiply this by 5 thence required these instructions: LOAD D, 5; MUL A, D; which left 35 in A. No one would label “transparent” this expression of computational flow.

¿How to make it so? The first idea was to associate A and D with the top two registers on an argStack; and replace the various LOAD instructions with a single PUSH instruction, like this: PUSH 3; PUSH 4; ADD; PUSH 5; MUL; which generated this stack sequence: 3; 3 4; 7; 7 5; 35. This was more transparent, and not quite so tedious. (Top of stack is in entry order, on the right.)

The next idea was that nouns should automatically push themselves onto the argStack; and verbs should be denoted in pithy ways:

3 4 + 5 *

This finally is flowing expression of its flowing computation. It is just as transparent as $(3+4)*5$ —more transparent when you recognize that nouns must be present before verbs can operate on them. This notation, in other words, corresponds directly to actual computation—no parsing required; no parenthesis required; you and the machine are in harmony.

Stack-based harmony allows you to transparently analyze expressions that fail to compute properly; but it comes with a downside: You must think like a stack—that was the fatal flaw in Forth. Specifically, you must constantly manipulate stack-resident nouns into the top stack registers using drop, roll, dup, pick ... to get verbs to operate on them.

That is easy for binary verbs like + and *, which immediately consume the top two nouns and leave their result in their place; but it is usually difficult for verbs with more than two or three arguments. The solution is to carefully stash pertinent args on the heap; and put them on the argStack just when they are needed, in proper order.

This requires giving nouns, and even verbs, context-dependent names that can be invoked at will. These names, if chosen carefully, help generate self-documenting code as a bonus, the goal of literate programming.

Which is where box-nested namespaces come in. A programmer would never write `3 4 + 5 *` because it can be pre-computed in any context as 35; but she might write `apples oranges + baskets *` to discover how many pieces of fruit are in a collection of picnic baskets (as a simple-minded example).

(Reading that kind of flow in linear prose like this, and composing it in linear prose, is somewhat baffling. A flow has a intrinsic branching structure whose prose sequence flattens, and fails to display. Happily, NestFlow does display the branches explicitly (they emerge at the verbs). With them in full view, flowing expression takes only a few days to start liking; and then about a month to come to love; absolutely love, in my experience.)

In a different context a programmer might write `chocoCherries mintyBears + baskets *` to discover how many pieces of candy are in a collection of party baskets. The picnic context and the party context use names that are not relevant to each other, but they share the basket context whose names (baskets at least) are relevant to both.

In other words, the picnic scope and the party scope are distinct, but both nested within the basket scope. Such scope has traditionally been delimited using curly braces. Unfortunately, its nesting is nearly impossible to discern unless augmented by disciplined indentation. That led to the idea that indentation alone should establish scope, as done in most recent languages: Python, Ruby, CoffeeScript ...

This discipline is a significant advance because it enforces visual nesting of scope boundaries—the code won't compile otherwise. However, discovering the computational structure of the code still requires the same kind of parsing

that curly braces require.

That structure can be automatically generated—and the nesting boundaries can be much much much better exposed—by using the Document Object Model, the much-maligned DOM, to represent box-nested namespaces. Unfortunately it was launched near the end of the last millennium with a verbose and tedious API. Fortunately it is getting lean at the beginning of this millennium—it has become almost a pleasure to articulate.

The DOM almost appears, in retrospect, to have been specifically concocted to be an efficient framework for articulating programming languages. Said technically, it establishes an annotated syntax tree that is not only visually obvious to the programmer, but also computationally straightforward for the machine—it provides genuine harmony between programmer and programme.

With it, you will effectively be writing simple machine code that is nonetheless immediately understandable by a human. Or said conversely, simple human prose that is immediately computable by a machine. This is especially clear for NestFlow's control structures, of which there are only two: skip-if and verb self-invoking.

Since skip-if is so pervasive in NestFlow, it is abbreviated as skif, like so: skif=, skif!= skifN, skif>, skifS, skif< ... meaning skip-if-equal, skip-if-not-equal, skip-if-none, skip-if-more, skip-if-some, skip-if-less ... and so on. The fundamental branching distinction in NestFlow is the natural and very expressive some-none distinction, S versus N

The primordial version of this distinction emerged in the nascent C language, where Nothing was initially bit-pattern 00 . . 0, and Something was any other bit-pattern. The None distinction steadily evolved to include No-number, No-object, No-string, No-definition, ... etc; which of course caused Something to become a tiny bit smaller: none of those various Nothings.

As that occurred, it became necessary to have a generic representation of all of the emergent Nothings, commonly called false. This naturally necessitated a converse representation of the residual Somethings, called true. These linguistic distinctions became a computational mess; but a different mess in

every different language.

The keywords `true`, `false`, `if`, `else`, `for`, `while`, ... are abstract appurtenances of English that obscure concrete flowing computation—they disrupt the harmony between human and machine by interposing artificial prose distinctions that must be compiled away to get computation to flow.

When that is done, you are left with bare skifs for forward branches. However, defining them in a way that is transparent to a human, yet efficient for a machine requires that the Some-None true-false mess finally be cleaned up, one of the primary goals of NestFlow.

A common misconception is that, for human comprehension, computation should be expressed in one of the natural languages engendered by our social evolution. That is a strange idea because our evolution only became seriously engaged with computation some seventy years ago; just a blink in the hundred-thousand year history of *Homo sapiens*; of genus *Homo*, itself three to five million years old. Seventy years, by comparison, is more of a nano-blink than a blink; not enough time for natural language to have evolved good computational expressiveness.

But that process is now underway. The most comprehensible computational word that has emerged so far is `return`. Every modern computer language has it; and every programmer immediately understands it—it is a primordial flow-altering word; it is the beginning of a good computational lexicon. For the same reason, every programmer can also immediately understand a skif.

(My coder friends testify—and I can also testify—that during their first year or two of programming, getting nesting and indentation right on complex if-else blocks was really tricky owing to dangling else clauses. Then it became merely tricky. In NestFlow, contrariwise, there is no trickiness about skipping over a visual block, not even one that itself contains skipped nested visual blocks, that contain skipped nested visual blocks... To say this differently, scope never dangles in NestFlow, unlike virtually all if-else languages.)

From a programmer's perspective a skif functions as a flow-altering verb that consumes either the remaining arg on the stack for a some-none skif; or else the remaining two args for a more-less skif, or other binary ones like `equal` or

not-equal. If the skif condition is satisfied, the box immediately after is skipped over, otherwise it is entered.

From the machine's perspective, a skif verb is nothing more than a comparison followed by a conditional forward branch. This is precisely how a compiler generates its if-else code, so NestFlow's source code is not only largely pre-parsed, it is almost pre-compiled too. What remains is mostly direct transpilation to machine code by, for example, replacing a skif with the compare-branch opcode pertinent to that machine.

¿Do I need to mention how transparent this makes debugging control flow?

Now about self-invoking verbs, which are NestFlow's backward branches. They operate the same way as any verb: by consuming previous nouns on the argStack. For this to be effective, a verb must either consume a fixed number of nouns (like + and * do); or else it must be prepended in some pithy way to indicate how many arguments it consumes.

(Aside: Also to be effective, a verb must be able to be deactivated so that a subsequent verb can commandeer it as an argument; to enhance or specialize it in some way, or deploy it as a helper; or perhaps even to compose it into a new verb to be returned as an arg.)

If a verb self-invocation is a tail-call, EcmaScript's new optimization mandates that the call be a backward branch with updated arguments—an efficient loop in other words. Otherwise the call would tediously generate another stack frame, and another, and another ... This could easily consume much time and memory, perhaps even all memory; which is why this technique was not feasible before, except in better-disciplined languages like Scheme ...

... a language that, incidentally, Brendan Eich, JavaScript's designer, initially wanted to use in the browser. Unfortunately, his management felt it was too esoteric for naifs; so we got stuck with Java-like clunkiness in the browser until recently.

Sophisticates are unlikely to agree with all of the design choices I made; so NestFlow was intentionally developed to be evolvable by smart coders. To

play with it offline, download it from github, and then serve it from a local server, as described in the Playing Around section of the Final Thoughts chapter

Best of all, you can craft the language to suit yourself. If most other coders like your enhancements, NestFlow will evolve in its own sweet time. If they don't like your enhancements, you will at least have something that suits you, if no one else.

This malleability is available because NestFlow is still in a nascent alpha state, un-rigidified by a language committee so far. Altho I have been playing with its ideas for half a century now, it was only a decade ago that I began to implement them as a hobby. I had high hopes to have a fully functional language by now. Ha.

My reach exceeded my grasp; but during the time NestFlow was within my grasp, it was the most fun, almost, that I have ever had without laughing. It is now within your grasp if you wish. Have fun.

Flowing ExpressionLet us begin with the extremely simple computational flow mentioned in the Preface. To get it going, hop over to the Playing Around section in the Final Thoughts chapter. After launch, you will see this:

Welcome: try $3\ 4 + 5 *$

This is not an image—it is simply the welcoming HTML generated by NestFlow. (To see that, try resizing this view.) As you compose your program, the generated HTML exposes the program structure to you and to the machine. Its raw `textContent`, slightly enhanced by some structural HTML, will be your NestFlow source code.

(Aside about the boxed HTML: It displays uniformly and properly in browsers and eBook readers, which all adhere to W3C standards. Amazon's Kindle devices and apps deviate from those standards, so what you see on them may be somewhat different from what you would see when coding in NestFlow on a browser. This is truly unfortunate because those differences may lose some important distinctions; but they are generally close enough not to confuse you. For maximum clarity, follow along on your browser.)

The source text is structured to be surprisingly readable in a text editor, even tho it is intended to be read and edited in the DOM. The two forms are directly interconvertible.

The text form is lean because it streamlines the customary control Englishing; but keeps the crucial namespace Englishing (or Frenching, or Chinesing ...) Its leanness is hinted at in the top infoNest, which exhibits a fundamental philosophy of NestFlow: display only the crucial bare-bones information needed in the current context. Here, that is to begin coding in the gray-bordered outer nest—the oNest—below it.

To do that, click within the tan editNode nestled in the blue-green flo. That node always proceeds at the downstream end of a flo. When a flow becomes complete (with a w), the editNode exits it, ready to be embedded to a new flo. A flo becomes complete whenever its argStack becomes empty; typically by having its remaining argument consumed by a name, or by a skif; as will become clear.

To code in NestFlow, you enter one word after another in the editNode. Whatever word you enter will be a meaningless string of characters until you press the spacebar. Then, if NestFlow can make sense of it, it leaves the editNode and augments the flo, structured and colored according to its meaning. The emptied editNode then advances downstream, ready for you to enter another word.

You can't code in your Kindle device (yet—you could in an EPUB3 device, in a constrained way), but you can in your browser. Begin by entering “3 space” and “4 space”, after which you see this:

argStack: 3 4

3o 4o

You just entered two words, 3 and 4, using the editNode, which has advanced downstream as evidenced by its canoe-tan color. (And also evidenced by its blinking cursor, not displayed in these passive views. Moreover, some browsers outline the editNode to indicate that it is contenteditable.) The editNode may be placed on any flow, as you shall see; and it disciplines your completed flows to always be syntactically correct, a wonderful feature of NestFlow's pre-parsed simplicity.

¿Did you notice that the words 3 and 4 changed—after they entered the flow—from dull gray in the editNode to the color of numbers? (Namely bluish purple, which you may change in the CSS file if you wish). NestFlow has accepted your prose—it has meaning now.

The green o's that appeared after the 3 and 4 are o-utput indicators. They indicate that these numbers have been placed on the argStack (in that order) ready to be consumed by a subsequent verb. The infoNest message area confirms it—that is the information crucial for this particular context. It is redundant here but it won't be in the very next step, which is to enter "+ space". Doing so launches the rudiments of flowing expression:

argStack: 7

3 4 +o

The output indicators on 3 and 4 have disappeared because those two numbers were consumed by the + verb, so are no longer on the argStack. What is on the stack now is the output of that verb, namely 7, as the infoNest indicates.

That output is again marked by a green o-utput indicator. Whenever you see that indicator, you know that a subsequent verb can consume it as an argument. This suggests, and properly so, that NestFlow considers nouns to be nullary verbs (those having no arguments) that generate a fixed output, their value.

The output indicators that had been on nullary 3 and 4 have been replaced by gray-dash argTies. When read horizontally from left to right, they indicate what the verb to their right has consumed, as you see. When read vertically up, they indicate the values that were consumed. Let us call such a horizontal display of args into its verb an invocation by that verb.

Notice that the + verb has the same purple color as a number because that is what it outputs; but its dynamic character is indicated by italics. All verbs are indicated by italics; all nouns are indicated by regular typeface. All words—verbs and nouns—have their output further indicated by their color.

As a slight aside, outputs in NestFlow have potential to be any, a type marked in gray. The inclusion of any in NestFlow is not merely for convenience and compatibility with JavaScript, tho that is a factor. More crucially, it is necessary for handling NestFlow's vacuties, which had been collectively called none in the Preface.

NestFlow attempts to unify all of JavaScript's vacuities: 0, NaN, undefined, false, null, ""; and even [], {}. It does so by assigning the six emboldened values to the following constants, respectively: **N**, **Nn**, **Nd**, **Nt**, **No**, **Ns**; meaning Nothing, No-number, No-definition, No-truth, No-object, No-string.

Each of these has traditionally been viewed and implemented as a different kind of nothing, an idea that has landed us in a mess. It violates Sturgeon's revelation that nothing is absolutely so, meaning that nothing has no properties whatsoever. To harmonize with that, NestFlow views and implements these vacuities quite differetly: as nothing in different contexts. This cleanly separates the nothingness from its context, which you can often ignore. When context is important, you can deal with it separately. You'll see.

The context for each of the previous nullities except **Nd** is given by the type associated with it. **Nd** can occur in any context in NestFlow, so it is associated with the type any.

For consistency, NestFlow should also assign **[]** as **Na** and **{}** as **No** (again), meaning No-array and No-object; but that is impossible because of the way JavaScript implements each as a new distinct empty array and a new distinct

empty object. In particular, altho it is true that `"" === ""`, it is not true that `[] === []`, or that `{ } === { }`, or even that `NaN === NaN` as you may check.

The Fixing Nothing chapter shall rassle with all of this. In the meantime, the DOM and CSS make most of the other Nullary distinctions and structure almost straightforward to implement, and easy for you to tweak to your own preferences, a fact that delights me.

NestFlow's structure establishes a computational tree that flows right in the view. It grows in the opposite way a natural tree does, from noun leaves, which converge to new branches, which eventually coalesce to a root answer.

So NestFlow's code structure is more like a river whose various forks eventually flow into a main flow; whose confluences are the verbs. To observe all of this further, let us take the next step and enter "5 space" to generate a new headwater on our flowing river of computation:

```
argStack: 7 5
```

```
3 4 +o 5o
```

Now you see two outputs again, one from the verb `+`, and one from the noun `5`. The infoNest tells you the verb output is `7`, and of course the noun output is `5`. These outputs are ready to be consumed by a subsequent verb, which will generate a new confluence in the river. We have decided it will be `*`. After you enter that, everything changes again:

```
argStack: 35
```

```
3 4 + 5 *o
```

You may have anticipated most of this: The two previous outputs on the stack have been consumed by `,` *and replaced by argTies to the `+` output and the `5` output. There is now only one value on the stack, `35`, output by `.`* Obvious, right?

What might not be so obvious is that the invocation by `*` has descended below the one by `+`. It need not have: they could have been placed on the

same level and still each have been individually comprehensible as argTies flowing into the verb on their right.

However, their individuality would not have been immediately obvious. NestFlow strives to be obvious, so adjacent verb invocations are always placed on different levels, a bend in the river. The * invocation could have been (and would have been, for tidiness) above the + invocation if there had been enough room. But there wasn't—the number 5 was in the way.

Speaking of invocations, ¿Are there different ones that produce the same result? Well, you know that $(3+4)*5$ equals $5*(3+4)$. You also know that, in either case, you must do the addition first to get the answer right—that is the whole purpose of the parenthesis. So you might suppose that the parenthesis-free way we just did it, $3\ 4\ +\ 5\ *$, is the only way it will work in a flowing language like NestFlow.

Fortunately that is not true because you can pre-stash nouns on the argStack, thereby deferring their consumption until later. To try this, you will need to recompose your flo like so: With the editNode selected, press “enter” (or “return”), which is the unflow-reflow key. This dumps the entire textContent of that flo into its editNode. Select all of that by pressing command-a, and then press delete (or backspace). Finally, enter “5 space 3 space 4 space” to stash these numbers on the argStack, like so:

argStack: 5 3 4

5o 3o 4o

Notice that this order corresponds to the order in $5*(3+4)$; an order that allows you to now do the addition first. After you enter that verb, it consumes the top two numbers on the argStack—the two numbers to its immediate left—as you see:

argStack: 5 7

5o 3 4 +o

The + verb has ingested 3 and 4, and output 7, which replaced them on top of the argStack. The 5 on that stack was untouched because it was at the bottom, where it remains. That is how you stash things for later use: you put them on the bottom of the stack by invoking them first.

Now there are only two nouns on the argStack, so any binary verb we invoke will ingest both. We invoke *:

argStack: 35

5 3 4 + *o

You see that this flo produced the same output as the previous one, namely 35. It reads somewhat like German: a pile of nouns followed by a sequence of verbs to deal with them. Which prompts this question: ¿Is one way better than the other? To answer, let's regenerate these two flows one after the other, cleanly separated by a simple beep. To do so, simply delete your code as previously done, and then enter the following code:

argStack: 35 35

3 4 + 5 o *beep* | 5 3 4 + o

Look first at the original flo on the left. Each verb invocation consumes the nouns to its immediate left contiguously, which makes the code immediately obvious. The flo on the right is not so clear—the *-invocation has to extract

its first argument upstream above the +-invocation, as you see. That generates a more turbulent, less obvious flo.

Smooth flo can often be achieved for arithmetic operations because they often commute, meaning that their arguments can be swapped to get them into invocation order. Few other verbs have that lovely symmetry; so you will sometimes need to pre-stash nouns on the stack for later use.

To minimize the need for that, a good strategy is to put selected nouns on the heap by naming them; and only place them on the stack when needed, in invocation order. Such names, when chosen carefully, generate self-documenting code, as mentioned in the Preface.

Notice that the beep verb separating the two computations consumed nothing. However, like all functions in JavaScript and NestFlow, it always returns something (in addition to presumably emitting a beep). That something is not anything of value, so this verb has the dull gray not-any color; and its nullary output has been intentionally blocked with a post-bar | (colored green because it too is an output indicator: no-output). This bar functions as a compose-time verb that prevents a return value from entering the arg stack.

Design musing—always returning somethingJavaScript was intended to be as accommodating as possible; and part of that is for every function to uniformly return one result, whether it is useful or not. This is not like earlier languages, which typically decline to output anything that is not useful.

Beep, in particular, always returns undefined (which NestFlow provisionally represents as Nd—No-definition). At first glance that may not seem useful; but it actually is in certain scenarios, as we shall see shortly. So, at least in this respect, NestFlow follows JavaScript's accommodating character.

My naive early inner-programmer thought that returning nothing of value was a design flaw, coming as I did from an initial background in C, Fortran, and Forth. I now think its uniformity is a stroke of genius—I like installing and seeing any requisite output-blocking because it emphasizes that this verb is being used only for its side effect, not its return value. This is a crucial distinction, well deserving such emphasis.

However, for flowing computation to work, this does mean that you will occasionally need to block a verb's output—sometimes even an otherwise useful output—to prevent it from being used.

This is true even in JavaScript itself, but the blocking maneuver is more obscure there: don't enclose the verb within argument parenthesis. In NestFlow, by contrast, you block more obviously: an explicit post-bar to keep it off the argStack. These contrasting maneuvers shall impose style restrictions on NestFlow when it is compiled (un-parsed, really) into JavaScript. The Becoming Functional chapter deals with this.

There is one final nuance about always returning something: It means that the return operator always takes exactly one argument, namely the value it returns. In JavaScript, this argument is implicitly undefined if it is not specified; but in NestFlow, the argument of return must always be specified; and for good reason, up next.

The verb return is very special: it is a flow terminator meaning that it must empty the argStack (just as name assignment must, and a skif must—you'll see). That is why it must always consume an argument: it can't empty the argStack otherwise.

Let us now rattle with beep's vacuous output to see how useless it usually is. To do so, you will need to go back upstream and reenter that verb without blocking it. Going upstream in NestFlow, however, is computationally as hard as paddling upstream in a river. Here is the problem: